Isolation Bearing Design

301 System Period

Example 04 - rivtbook

proj. 0004

Table of Contents

0.6 | section 1

CLOUGH, PENZIEN - Dynamics of Structures, page 178

Use flexibility formulation [1] (see page 182)

Also see [2]

calioPY Procedure Output - Example 6 This example illustrates arrays and plotting.

1. set up mass and stiffness arrays m = PL.array([[1.0,0,0],[0,1.5,0],[0,0,2.0]],float) [1] k1 = 600·PL.array([[1,-1,0.0],[-1,3,-2],[0,-2,5]],float) [2]

2. flexibility and dynamic matrix f = PL.inv(k1) [3] d = PL.inner(f,m) [4]

3. eigenvalues eigen = PL.eig(d) [5] evalu = eigen[0] [6] . evalu = [0.00474 0.00104 0.00047] nat_freq = 1/(evalu^.5) [7] . nat_freq = [14.52482 31.00868 46.12656]

4. normalize and scale eigenvectors evect = PL.array(eigen[1]) [8] . evect = [[-0.813 -0.739 0.273] [-0.527 0.449 -0.694] [-0.246 0.502 0.666]]

[py] for i in range(len(nat_freq)): [py] evectt = PL.transpose(evect) [py] evectt[i] = evectt[i]/evectt[i][0]

0.6 - 2 | Eigenvectors and Eigenvalues

normalized eigenvectors

[[1. 0.648 0.303] [1. -0.608 -0.679] [1. -2.542 2.44]] SUMMARY TABLES

mass matrix 1.0 0.0 0.0 0.0 1.5 0.0 0.0 0.0 2.0

stiffness matrix 600.0 -600.0 0.0 -600.0 1800.0 -1200.0 0.0 -1200.0 3000.0

flexibility matrix 0.0031 0.0014 0.0006 0.0014 0.0014 0.0006 0.0006 0.0006 0.0006

dynamic matrix 0.003 0.002 0.001 0.001 0.002 0.001 0.001 0.001 0.001

xx = NP.concatenate((evals,evect),1) yy = [“freq”,”level 3”,”level 2”,”level 1”] tt = NP.vstack((yy,xx)) . tt = [[‘freq’ ‘level 3’ ‘level 2’ ‘level [‘14.52482’ ‘1.0’ ‘1.0’ ‘1.0’] [‘31.00868’ ‘0.648’ ‘-0.608’ ‘-2.542’] [‘46.12656’ ‘0.303’ ‘-0.679’ ‘2.44’]]

eigenvectors and eigenvalues freq level 3 level 2 level 1 14.52482 1.0 1.0 1.0 31.00868 0.648 -0.608 -2.542 46.12656 0.303 -0.679 2.44

0.6 - 3 | Plot Mode Shapes

```
1. set up mass and stiffness arrays
[1] m = PL.array([[1.0,0,0],[0,1.5,0],[0,0,2.0]],float
[2] k1 = 600*PL.array([[1,-1,0.0],[-1,3,-2],[0,-2,5]

2. flexibility and dynamic matrix
[3] f = PL.inv(k1)
[4] d = PL.inner(f,m)

3. eigenvalues
[5] eigen = PL.eig(d)
[6] evalu = eigen[0] ?5
[7] nat_freq = 1/(evalu**.5) ?5

4. normalize and scale eigenvectors
[8] evect = PL.array(eigen[1]) ?


-pycode-
for i in range(len(nat_freq)):
evectt = PL.transpose(evect)
evectt[i] = evectt[i]/evectt[i][0]
-insert-

normalized eigenvectors
{] evectt ?3

 SUMMARY TABLES

-table-02 m ?4; mass matrix;

-table-02 k1 ?4; stiffness matrix;

-table-02 f ?4; flexibility matrix ;

-table-02 d ?3; dynamic matrix ;

-k-
# use reshape to transpose 1d array (list)

evals=PL.reshape(nat_freq, (len(nat_freq),1))
xx = NP.concatenate((evals,evect),1)
yy = ["freq","level 3","level 2","level 1"]
tt = NP.vstack((yy,xx)) ?
-table-01 tt ?3; eigenvectors and eigenvalues;
.
# initialize eigenvector array (need (x,1) shapes for plotting
ms = PL.shape(evect)
zz = PL.zeros((ms[0],1))
x1=PL.concatenate((evect,zz),1)
-pycode-
#plot mode shapes using pylab
y=PL.array([0,1,2,3])
m3=x1[2]*.35+3
```


```
m2=x1[1]*.35+2
m1=x1[0]*.35+1
m=PL.concatenate((m1,m2,m3))
PL.clf()
PL.plot(m1,y)
PL.plot(m2,y)
PL.plot(m3,y)
PL.xlim(.5,4.)
PL.xlabel('mode')
PL.ylabel('levels')
PL.title("Mode Shapes
gh\Penzien (page 178)")
PL.grid()
PL.savefig(_cpypath+"/figs/p178a.png")
```

---

[1] R.W. Clough and J. Penzien, *Dynamics of Structures*. New York, NY, USA:McGraw-Hill, 1975.

[2] Anil K.Anil K. Chopra, Dynamics of Structures: Theory and Applications toEarthquake Engineering. Englewood Cliffs, NJ, USA: Prentice Hall, 1995.